

1 Gerenciamento de Configuração e Versionamento

Este documento estabelece as diretrizes formais para a política de submissões (*commits*) e o controle de versões do software para o projeto de Engenharia de Software. A padronização visa garantir a manutenibilidade, a rastreabilidade e a consistência histórica do repositório de código.

1.1 Política de Commits (Conventional Commits)

Para manter o histórico de desenvolvimento claro, auditável e semântico, todas as mensagens de envio (*commits*) devem obrigatoriamente seguir a especificação *Conventional Commits*.

A estrutura padrão de uma mensagem de *commit* é definida por:

```
<tipo>(<escopo>): <mensagem curta em letras minúsculas>
```

O **tipo** identifica categoricamente a natureza da alteração realizada. Os tipos aceitos e padronizados para este projeto são:

- **feat**: Introdução de uma nova funcionalidade no sistema (ex: nova regra de negócio, nova rota de API).
- **fix**: Resolução de um problema ou correção de um *bug* encontrado no código.
- **docs**: Alterações exclusivas em arquivos de documentação (ex: `README.md`, manuais, comentários Javadoc).
- **test**: Adição, modificação ou correção de cenários de testes unitários (utilizando a biblioteca JUnit).
- **refactor**: Modificações na estrutura interna do código que melhoram sua legibilidade ou arquitetura, sem alterar o comportamento final esperado e sem adicionar novas funcionalidades.

Exemplos Práticos de Commits

- `feat(gof): implementa a interface e as classes concretas do padrao strategy`
- `test(junit): elabora os 15 casos de teste estruturados para o service`
- `fix(db): corrige erro de tabela nao encontrada no ambiente de stub`
- `docs(readme): atualiza as instrucoes de execucao do prototipo`
- `refactor(accounting): simplifica o fluxo de calculo de impostos internos`

1.2 Controle de Versão Semântico (SemVer 2.0.0)

O gerenciamento de lançamentos e controle de *releases* do produto de software utilizará rigorosamente a especificação **Versionamento Semântico 2.0.0** (*Semantic Versioning*).

O número da versão é estruturado no formato **X.Y.Z**, mapeado como:

$$\mathbf{X.Y.Z} \rightarrow \mathbf{MAJOR . MINOR . PATCH}$$

Os critérios de incremento de cada componente do número de versão estão diretamente vinculados à política de *commits* adotada pelo time:

1. **PATCH (Z)**: Incrementado quando são realizadas correções de erros retrocompatíveis.
 - *Gatilho*: Ocorrência de um ou mais commits do tipo **fix**.
 - *Exemplo*: A versão atual evolui de 1.0.0 para 1.0.1.
2. **MINOR (Y)**: Incrementado quando novas funcionalidades são adicionadas ao software de forma retrocompatível, ou seja, sem quebrar as integrações existentes.
 - *Gatilho*: Ocorrência de commits do tipo **feat**.
 - *Exemplo*: A versão atual evolui de 1.0.1 para 1.1.0.
3. **MAJOR (X)**: Incrementado quando alterações drásticas ou quebras de compatibilidade na API/arquitetura são introduzidas no sistema, ou na entrega de marcos conceituais importantes do projeto (*milestones*).
 - *Gatilho*: Alterações estruturais profundas ou entregas de versões estáveis homologadas pelo grupo.
 - *Exemplo*: A entrega final do primeiro módulo evolui o sistema para 1.0.0. Uma reestruturação massiva da persistência altera de 1.1.2 para 2.0.0.

Tabela 1: Matriz de Correlação: Tipo de Alteração vs. Impacto no SemVer

Tipo de Commit (Conventional)	Impacto Arquitetural	Incremento SemVer
fix	Correção interna estável	PATCH (X.Y.Z+1)
docs / test / refactor	Sem alteração no produto final	Nenhum (Mantém X.Y.Z)
feat	Nova funcionalidade modular	MINOR (X.Y+1.Z)
Mudança Estrutural / Marco Técnico	Quebra de compatibilidade ou entrega	MAJOR (X+1.Y.Z)